"You see," Bill continues, "when a class includes a module, it gets the module's instance methods—*not* the class methods. Class methods stay out of reach, in the module's singleton class."

"So, how did you find a solution?" you ask. "Oh, I didn't," Bill replies, blushing. "I just asked for the solution on a mailing list, like everybody else does. But maybe *you* can find a solution." Think about the object model and singleton classes. How would you modify the code that you just looked at so that it works as expected?

Quiz Solution

The solution to this quiz is simple and subtle at the same time. First, define `my_method` as a regular instance method of `MyModule`. Then include the module in the singleton class of `MyClass`.

class_definitions/module_trouble_solution.rb

```
module MyModule
  def my_method; 'hello'; end
end

class MyClass
  class << self
    include MyModule
  end
end

MyClass.my_method   # => "hello"
```

`my_method` is an instance method of the singleton class of `MyClass`. As such, `my_method` is also a class method of `MyClass`. This technique is called a *Class Extension*.

Spell: Class Extension

"That's brilliant," Bill says. "What about trying the same trick on a regular object instead of a class?"

Class Methods and include()

Reviewing Class Extensions, you can define class methods by mixing them into the class's singleton class. Class methods are just a special case of Singleton Methods, so you can generalize this trick to any object. In the general case, this is called an *Object Extension*. In the following example, `obj` is extended with the instance methods of `MyModule`:

Spell: Object Extension

class_definitions/module_trouble_object.rb

```
module MyModule
  def my_method; 'hello'; end
end
```